

[VN] Virtual DOM trong React là gì?

[EN] What is the Virtual DOM in React?

[VN] Virtual DOM trong React là một bản sao của DOM thật được lưu trong bộ nhớ để giúp cập nhật giao diện nhanh hơn. Khi dữ liệu trong ứng dụng thay đổi, React sẽ tạo một Virtual DOM mới và so sánh nó với Virtual DOM cũ (quá trình này gọi là diffing). Sau đó React chỉ cập nhật những phần thật sự thay đổi lên DOM thật của trình duyệt thay vì render lại toàn bộ trang. Nhờ vậy, ứng dụng chạy nhanh hơn và hiệu quả hơn.

[EN] The Virtual DOM in React is a lightweight copy of the real DOM stored in memory to make UI updates faster. When the application data changes, React creates a new Virtual DOM and compares it with the previous one (this process is called diffing). React then updates only the parts that actually changed in the real browser DOM instead of re-rendering the whole page. Because of this, the application runs faster and more efficiently.

[VN] Cơ chế diffing (reconciliation) của React hoạt động như thế nào?

[EN] How does React's diffing (reconciliation) mechanism work?

[VN] Cơ chế diffing (reconciliation) trong React là quá trình React so sánh Virtual DOM mới với Virtual DOM cũ để tìm ra những phần đã thay đổi. React sử dụng một thuật toán tối ưu với một số giả định, ví dụ: nếu hai element có type khác nhau thì React sẽ tạo lại toàn bộ node đó, còn nếu type giống nhau thì React chỉ cập nhật những thuộc tính thay đổi. Khi xử lý danh sách (list), React sử dụng key để xác định phần tử nào được thêm, xóa hoặc thay đổi vị trí. Sau khi tìm được sự khác biệt, React sẽ cập nhật những phần cần thiết lên DOM thật để giảm số lần thao tác với trình duyệt và giúp ứng dụng chạy nhanh hơn.

[EN] The diffing (reconciliation) mechanism in React is the process where React compares the new Virtual DOM with the previous Virtual DOM to find what has changed. React uses an optimized algorithm with some assumptions, for example: if two elements have different types, React will recreate that node completely, but if the type is the same, React only updates the changed attributes. When working with lists, React uses keys to identify which items are added, removed, or moved. After finding the differences, React updates only the necessary parts in the real DOM, which reduces browser operations and makes the application faster.

[VN] Sự khác biệt chính giữa Virtual DOM và DOM thật là gì?

[EN] What is the main difference between the Virtual DOM and the real DOM?

[VN] Sự khác biệt chính giữa Virtual DOM và DOM thật trong React là cách chúng được sử dụng và hiệu suất cập nhật. DOM thật là cấu trúc cây mà trình duyệt dùng để hiển thị giao diện HTML, và mỗi lần thay đổi trực tiếp vào DOM thật thường tốn nhiều tài nguyên và chậm hơn. Virtual DOM là một bản sao nhẹ của DOM thật được lưu trong bộ nhớ. Khi

dữ liệu thay đổi, React sẽ cập nhật Virtual DOM trước, sau đó so sánh với phiên bản cũ để tìm ra sự khác biệt và chỉ cập nhật những phần cần thiết lên DOM thật. Nhờ vậy, việc cập nhật giao diện trở nên nhanh và hiệu quả hơn.

[EN] The main difference between the Virtual DOM and the real DOM in React is how they are used and their update performance. The real DOM is the tree structure used by the browser to display HTML on the screen, and directly changing the real DOM can be slow and resource-intensive. The Virtual DOM is a lightweight copy of the real DOM stored in memory. When data changes, React updates the Virtual DOM first, then compares it with the previous version to find the differences and updates only the necessary parts in the real DOM. This makes UI updates faster and more efficient.

[VN] Tại sao key lại quan trọng khi render danh sách trong React?

[EN] Why are keys important when rendering lists in React?

[VN] Trong React, key rất quan trọng khi render danh sách vì nó giúp React xác định chính xác phần tử nào đã thay đổi, được thêm vào hoặc bị xóa. Khi React thực hiện quá trình diffing (reconciliation), key được dùng để nhận diện từng phần tử trong danh sách. Nếu không có key hoặc key không ổn định (ví dụ dùng index), React có thể cập nhật sai phần tử hoặc render lại nhiều phần không cần thiết, làm giảm hiệu suất và có thể gây lỗi giao diện. Vì vậy, nên dùng một giá trị unique và ổn định làm key, ví dụ như id của dữ liệu.

[EN] In React, keys are important when rendering lists because they help React identify which elements have changed, been added, or removed. During the diffing (reconciliation) process, keys are used to uniquely identify each element in the list. If keys are missing or unstable (for example using the index), React may update the wrong elements or re-render more parts than necessary, which can reduce performance and sometimes cause UI bugs. Therefore, it is best to use a unique and stable value as a key, such as the id of the data.

[VN] Nếu dùng index làm key trong list, sẽ gặp những vấn đề gì?

[EN] What problems arise when using index as a key in a list?

[VN] Trong React, nếu dùng index làm key khi render danh sách, có thể gây ra một số vấn đề khi danh sách thay đổi như thêm, xóa hoặc sắp xếp lại phần tử. Vì index phụ thuộc vào vị trí của phần tử trong mảng, khi vị trí thay đổi thì key cũng thay đổi, khiến React nghĩ rằng phần tử cũ đã bị thay thế bằng phần tử mới. Điều này có thể dẫn đến việc cập nhật sai dữ liệu, mất state của component, hoặc render lại nhiều phần không cần thiết. Vì vậy, nên dùng một key ổn định và duy nhất như id của dữ liệu thay vì dùng index.

[EN] In React, using the index as a key when rendering a list can cause problems when the list changes, such as when items are added, removed, or reordered. Since the index

depends on the position of an item in the array, the key will change when the position changes. This can make React think the old element was replaced by a new one. As a result, it may update the wrong data, lose the component's state, or re-render unnecessary parts. Therefore, it is better to use a stable and unique key, such as the id of the data, instead of the index.

[VN] Ví dụ cụ thể về trường hợp key bị sai dẫn đến bug giao diện?

[EN] Specific example of a case where an incorrect key causes a UI bug?

[VN] Trong React, một ví dụ dễ hiểu về bug khi dùng index làm key là danh sách todo có nút xóa. Giả sử có 3 item: A, B, C và key lần lượt là 0, 1, 2. Nếu người dùng xóa item A, danh sách sẽ trở thành B, C và index mới sẽ là 0, 1. React có thể nghĩ rằng item B là item A cũ (vì cùng key 0), nên dữ liệu hoặc state của component có thể bị giữ sai. Kết quả là giao diện có thể hiển thị dữ liệu không đúng với item thực tế.

[EN] In React, an easy example of a bug when using the index as a key is a todo list with a delete button. Imagine there are 3 items: A, B, C with keys 0, 1, 2. If the user deletes item A, the list becomes B, C and the new indexes are 0, 1. React may think item B is the old item A (because they share the same key 0), so the component's data or state can be reused incorrectly. As a result, the UI may display data that does not match the actual item.

[VN] Controlled component trong React là gì?

[EN] What is a controlled component in React?

[VN] Trong React, controlled component là một component mà giá trị của input (như input, textarea, select) được quản lý bởi state của React thay vì do DOM tự quản lý. Điều này có nghĩa là mỗi khi người dùng nhập dữ liệu, sự kiện onChange sẽ cập nhật state, và state đó lại được dùng để hiển thị giá trị trong input thông qua thuộc tính value. Nhờ vậy, React luôn kiểm soát dữ liệu của form, giúp dễ kiểm tra dữ liệu, validate và xử lý logic trong ứng dụng.

[EN] In React, a controlled component is a component where the value of form inputs (such as input, textarea, or select) is controlled by the React state instead of the DOM itself. This means when the user types something, the onChange event updates the state, and that state is then used to display the value in the input through the value attribute. Because of this, React fully controls the form data, which makes it easier to validate data and handle application logic.

[VN] Uncontrolled component là gì và cách sử dụng?

[EN] What is an uncontrolled component and how is it used?

[VN] Trong React, uncontrolled component là component mà giá trị của input được quản lý trực tiếp bởi DOM thay vì bởi state của React. Điều này có nghĩa là React không lưu giá trị input trong state, mà khi cần lấy dữ liệu thì ta truy cập trực tiếp từ DOM bằng ref. Cách sử dụng thường là tạo một ref bằng useRef, gắn ref đó vào input, và khi cần xử lý (ví dụ khi submit form) thì đọc giá trị từ ref.current.value. Cách này đơn giản hơn trong một số trường hợp, nhưng React sẽ ít kiểm soát dữ liệu hơn so với controlled component.

[EN] In React, an uncontrolled component is a component where the value of an input is managed directly by the DOM instead of React state. This means React does not store the input value in its state, and when the value is needed, it is accessed directly from the DOM using a ref. The common way to use it is to create a ref with useRef, attach it to the input element, and read the value from ref.current.value when needed, such as when submitting a form. This approach is simpler in some cases, but React has less control over the form data compared to controlled components.

[VN] Khi nào nên dùng controlled component, khi nào nên dùng uncontrolled?

[EN] When should you use a controlled component, and when should you use an uncontrolled component?

[VN] Trong React, nên dùng controlled component khi cần React kiểm soát dữ liệu của form, ví dụ như khi cần validate dữ liệu, kiểm tra input theo thời gian thực, hoặc xử lý logic dựa trên giá trị người dùng nhập. Vì giá trị được lưu trong state nên React có thể dễ dàng quản lý và cập nhật giao diện. Ngược lại, uncontrolled component nên dùng khi form đơn giản và chỉ cần lấy dữ liệu khi submit, vì nó cho phép DOM tự quản lý giá trị input và chỉ lấy dữ liệu bằng ref khi cần. Cách này thường viết code nhanh và đơn giản hơn nhưng React sẽ ít kiểm soát dữ liệu hơn.

[EN] In React, controlled components should be used when React needs to control the form data, for example when you need validation, real-time input checking, or logic based on user input. Because the value is stored in React state, it is easier for React to manage and update the UI. On the other hand, uncontrolled components are useful for simple forms where you only need the data when submitting the form. In this case, the DOM manages the input value and the data is accessed using a ref when needed. This approach can make the code simpler, but React has less control over the data.

[VN] SyntheticEvent trong React là gì?

[EN] What is a SyntheticEvent in React?

[VN] Trong React, SyntheticEvent là một đối tượng sự kiện được React tạo ra để xử lý các sự kiện như click, change hoặc submit theo cách giống nhau trên mọi trình duyệt. Thay vì dùng trực tiếp sự kiện gốc của trình duyệt (native DOM event), React bọc nó trong

SyntheticEvent để cung cấp một API thống nhất và dễ sử dụng hơn. SyntheticEvent có các thuộc tính và phương thức giống với sự kiện thật, ví dụ preventDefault() hoặc stopPropagation(), giúp lập trình viên xử lý sự kiện một cách nhất quán và hiệu quả.

[EN] In React, SyntheticEvent is an event object created by React to handle events such as click, change, or submit in a consistent way across all browsers. Instead of using the browser's native DOM event directly, React wraps it inside a SyntheticEvent to provide a unified and easier-to-use API. SyntheticEvent has properties and methods similar to the real event, such as preventDefault() or stopPropagation(), which help developers handle events in a consistent and efficient way.

[VN] SyntheticEvent khác native DOM event ở những điểm nào?

[EN] How does a SyntheticEvent differ from a native DOM event?

[VN] Trong React, SyntheticEvent khác native DOM event ở một vài điểm chính. SyntheticEvent là một lớp wrapper do React tạo ra để cung cấp API sự kiện giống nhau trên mọi trình duyệt, giúp code dễ viết và dễ bảo trì hơn. Ngoài ra, React quản lý các sự kiện này thông qua event delegation, nghĩa là React lắng nghe sự kiện ở cấp cao hơn thay vì gắn listener vào từng phần tử. Trước đây SyntheticEvent còn dùng cơ chế event pooling để tái sử dụng object nhằm tối ưu hiệu năng, trong khi native DOM event là sự kiện thật do trình duyệt tạo ra và hoạt động trực tiếp trên DOM.

[EN] In React, SyntheticEvent is different from a native DOM event in several ways. SyntheticEvent is a wrapper created by React to provide a consistent event API across different browsers, which makes the code easier to write and maintain. React also manages these events using event delegation, meaning it listens for events at a higher level instead of attaching listeners to every element. In the past, SyntheticEvent also used event pooling to reuse objects for better performance, while a native DOM event is the real event created directly by the browser and works directly with the DOM.

[VN] Tại sao React cần dùng SyntheticEvent thay vì event native?

[EN] Why does React use SyntheticEvent instead of native events?

[VN] Trong React, React sử dụng SyntheticEvent thay vì native event để đảm bảo cách xử lý sự kiện giống nhau trên mọi trình duyệt. Các trình duyệt có thể có sự khác biệt nhỏ trong cách hoạt động của event, nên React tạo ra SyntheticEvent như một lớp wrapper để cung cấp API thống nhất. Ngoài ra, React còn dùng cơ chế event delegation, nghĩa là React lắng nghe sự kiện ở cấp cao hơn thay vì gắn listener vào từng phần tử, giúp giảm số lượng event listener và cải thiện hiệu năng của ứng dụng.

[EN] In React, React uses SyntheticEvent instead of native events to ensure consistent event handling across all browsers. Different browsers can have small differences in how events work, so React creates SyntheticEvent as a wrapper to provide a unified API. In addition, React uses event delegation, meaning it listens for events at a higher level instead of attaching listeners to every element. This reduces the number of event listeners and helps improve application performance.

[VN] useEffect trong React hoạt động như thế nào?

[EN] How does useEffect work in React?

[VN] Trong React, useEffect là một hook được dùng để thực hiện side effects trong function component, ví dụ như gọi API, cập nhật DOM, hoặc đăng ký event listener. useEffect sẽ chạy sau khi component được render. Nó nhận hai tham số: một function chứa logic cần thực hiện và một dependency array. Nếu dependency array rỗng ([]), effect chỉ chạy một lần khi component mount. Nếu có các giá trị trong mảng, effect sẽ chạy lại khi những giá trị đó thay đổi. useEffect cũng có thể trả về một function để cleanup khi component unmount hoặc trước khi effect chạy lại.

[EN] In React, useEffect is a hook used to perform side effects in function components, such as calling APIs, updating the DOM, or adding event listeners. useEffect runs after the component has been rendered. It takes two parameters: a function that contains the logic and a dependency array. If the dependency array is empty ([]), the effect runs only once when the component mounts. If the array contains values, the effect runs again whenever those values change. useEffect can also return a function used for cleanup when the component unmounts or before the effect runs again.

[VN] Dependency array trong useEffect có vai trò gì?

[EN] What is the role of the dependency array in useEffect?

[VN] Trong React, dependency array trong useEffect dùng để xác định khi nào effect cần chạy lại. Đây là một mảng chứa các biến hoặc state mà effect phụ thuộc vào. Khi giá trị trong mảng này thay đổi sau mỗi lần render, React sẽ chạy lại useEffect. Nếu dependency array rỗng ([]), effect chỉ chạy một lần khi component được mount. Nếu không truyền dependency array, useEffect sẽ chạy sau mọi lần render. Vì vậy dependency array giúp React kiểm soát khi nào cần thực hiện side effect và tránh chạy lại không cần thiết.

[EN] In React, the dependency array in useEffect is used to determine when the effect should run again. It is an array that contains variables or state values that the effect depends on. When any value in this array changes after a render, React will run the useEffect again. If the dependency array is empty ([]), the effect runs only once when the component mounts. If no dependency array is provided, useEffect runs after every render.

Therefore, the dependency array helps React control when side effects should run and avoids unnecessary executions.

[VN] Khi dependency array là [] thì useEffect chạy lúc nào?

[EN] When the dependency array is [] in useEffect, when does it run?

[VN] Trong React, khi dependency array là [], useEffect sẽ chỉ chạy một lần sau khi component được render lần đầu (mount). Điều này có nghĩa là effect sẽ không chạy lại khi component re-render vì không có dependency nào thay đổi. Cách này thường được dùng cho các tác vụ chỉ cần thực hiện một lần, ví dụ như gọi API lần đầu, khởi tạo dữ liệu hoặc đăng ký event listener.

[EN] In React, when the dependency array is [], useEffect will run only once after the component is rendered for the first time (mount). This means the effect will not run again on re-renders because there are no dependencies to track. This pattern is commonly used for tasks that should run only once, such as calling an API when the page loads, initializing data, or adding an event listener.

[VN] Khi không truyền dependency array thì useEffect chạy ra sao?

[EN] How does useEffect run when no dependency array is provided?

[VN] Trong React, nếu không truyền dependency array cho useEffect, thì effect sẽ chạy sau mỗi lần component render. Điều này có nghĩa là mỗi khi state hoặc props thay đổi và component re-render, useEffect sẽ được thực thi lại. Cách này có thể hữu ích trong một số trường hợp, nhưng nếu không cẩn thận có thể gây chạy effect quá nhiều lần hoặc thậm chí tạo vòng lặp render nếu bên trong effect có cập nhật state.

[EN] In React, if you do not provide a dependency array to useEffect, the effect will run after every render of the component. This means whenever state or props change and the component re-renders, useEffect will run again. This can be useful in some cases, but if not used carefully it may cause the effect to run too many times or even create a render loop if the effect updates the state.

[VN] Cleanup function trong useEffect dùng để làm gì?

[EN] What is the purpose of the cleanup function in useEffect?

[VN] Trong React, cleanup function trong useEffect được dùng để dọn dẹp hoặc hủy các tác vụ đã tạo trước đó nhằm tránh lỗi hoặc rò rỉ bộ nhớ. Cleanup function được trả về từ useEffect và sẽ chạy khi component unmount hoặc trước khi effect chạy lại khi dependency thay đổi. Ví dụ phổ biến là hủy event listener, clear interval/timer, hoặc hủy subscription. Nhờ vậy ứng dụng hoạt động ổn định và không giữ lại các tài nguyên không cần thiết.

[EN] In React, the cleanup function in `useEffect` is used to clean up or cancel previously created side effects to avoid errors or memory leaks. The cleanup function is returned from `useEffect`, and it runs when the component unmounts or before the effect runs again when dependencies change. Common examples include removing event listeners, clearing intervals or timers, or canceling subscriptions. This helps keep the application stable and prevents unnecessary resource usage.

[VN] Cho ví dụ thực tế về cleanup trong useEffect (subscription, timer...).

[EN] Give a practical example of cleanup in useEffect (subscription, timer...).

[VN] Trong React, một ví dụ thực tế về cleanup trong `useEffect` là khi sử dụng timer hoặc subscription. Ví dụ, nếu component tạo một `setInterval` để cập nhật dữ liệu mỗi vài giây, thì khi component bị unmount hoặc effect chạy lại, ta cần dùng cleanup function để gọi `clearInterval`. Nếu không làm vậy, timer vẫn tiếp tục chạy dù component đã bị xóa, có thể gây lỗi hoặc rò rỉ bộ nhớ. Vì vậy cleanup giúp dọn dẹp các tài nguyên không còn cần thiết.

[EN] In React, a common real example of cleanup in `useEffect` is when using a timer or subscription. For example, if a component creates a `setInterval` to update data every few seconds, when the component unmounts or the effect runs again, we should use a cleanup function to call `clearInterval`. If we do not do this, the timer will continue running even after the component is removed, which may cause errors or memory leaks. Therefore, cleanup helps remove resources that are no longer needed.

[VN] Tại sao cleanup chạy trước khi component unmount và trước khi effect chạy lại?

[EN] Why does cleanup run before a component unmounts and before the effect runs again?

[VN] Trong React, cleanup function chạy trước khi component unmount và trước khi `useEffect` chạy lại để dọn dẹp các side effect cũ trước khi tạo side effect mới. Điều này giúp tránh việc nhiều effect cùng tồn tại và gây lỗi, ví dụ như nhiều event listener, nhiều timer hoặc nhiều subscription hoạt động cùng lúc. Khi component unmount, cleanup giúp giải phóng tài nguyên và tránh rò rỉ bộ nhớ. Khi dependency thay đổi và effect chạy lại, cleanup đảm bảo effect cũ được hủy trước khi effect mới được tạo.

[EN] In React, the cleanup function runs before a component unmounts and before `useEffect` runs again to remove old side effects before creating new ones. This helps prevent multiple effects from existing at the same time, such as multiple event listeners, timers, or subscriptions running together. When the component unmounts, cleanup releases

resources and prevents memory leaks. When dependencies change and the effect runs again, cleanup ensures the previous effect is removed before the new one is created.

[VN] useMemo dùng để làm gì?

[EN] What is useMemo used for?

[VN] Trong React, useMemo là một hook dùng để ghi nhớ (memoize) kết quả của một phép tính để tránh phải tính toán lại không cần thiết mỗi lần component render. Khi component render lại, React sẽ chỉ tính lại giá trị bên trong useMemo nếu dependency trong mảng dependency thay đổi. Nếu không thay đổi, React sẽ dùng lại kết quả đã được lưu trước đó. Điều này giúp tối ưu hiệu năng, đặc biệt khi có các phép tính phức tạp hoặc tốn nhiều tài nguyên.

[EN] In React, useMemo is a hook used to memoize (remember) the result of a calculation so that it does not need to be recalculated on every component render. When the component re-renders, React will only recompute the value inside useMemo if the dependencies in the dependency array change. If they do not change, React will reuse the previously stored result. This helps improve performance, especially when the calculation is complex or expensive.

[VN] useCallback dùng để làm gì?

[EN] What is useCallback used for?

[VN] Trong React, useCallback là một hook dùng để ghi nhớ (memoize) một function để React không tạo lại function đó mỗi lần component render. Khi component render lại, React chỉ tạo function mới nếu giá trị trong dependency array thay đổi. Nếu dependency không thay đổi, React sẽ dùng lại function cũ. Điều này giúp tránh re-render không cần thiết, đặc biệt khi truyền function xuống các component con.

[EN] In React, useCallback is a hook used to memoize a function so that React does not recreate the function on every component render. When the component re-renders, React will only create a new function if the values in the dependency array change. If the dependencies stay the same, React reuses the previous function. This helps avoid unnecessary re-renders, especially when passing functions to child components.

[VN] Điểm khác biệt chính giữa useMemo và useCallback?

[EN] What is the main difference between useMemo and useCallback?

[VN] Trong React, điểm khác biệt chính giữa useMemo và useCallback là giá trị mà chúng ghi nhớ. useMemo dùng để ghi nhớ kết quả của một phép tính hoặc một giá trị, giúp tránh phải tính lại khi component render nếu dependency không thay đổi. Trong khi đó,

useCallback dùng để ghi nhớ một function, giúp React không tạo lại function đó mỗi lần render. Nói đơn giản, useMemo trả về một giá trị, còn useCallback trả về một function.

[EN] In React, the main difference between useMemo and useCallback is what they memoize. useMemo is used to memoize the result of a calculation or a value, so React does not need to recompute it if the dependencies do not change. On the other hand, useCallback is used to memoize a function, so React does not recreate that function on every render. In simple terms, useMemo returns a value, while useCallback returns a function.

[VN] Khi nào KHÔNG nên dùng useMemo hoặc useCallback?

[EN] When should you NOT use useMemo or useCallback?

[VN] Trong React, không nên dùng useMemo hoặc useCallback khi phép tính hoặc function rất đơn giản và không tốn nhiều tài nguyên. Việc sử dụng các hook này cũng có chi phí vì React phải lưu và so sánh dependency, nên nếu dùng quá nhiều có thể làm code phức tạp hơn mà không cải thiện hiệu năng. Ngoài ra, nếu component không gặp vấn đề re-render hoặc không có phép tính nặng, thì việc dùng useMemo hoặc useCallback thường không cần thiết.

[EN] In React, you should not use useMemo or useCallback when the calculation or function is simple and not expensive. These hooks also have a cost because React needs to store the value and compare dependencies, so overusing them can make the code more complex without improving performance. Also, if the component does not have re-render issues or heavy computations, using useMemo or useCallback is usually not necessary.

[VN] React.memo là gì và hoạt động như thế nào?

[EN] What is React.memo and how does it work?

[VN] Trong React, React.memo là một kỹ thuật tối ưu hiệu năng dùng cho function component để tránh render lại không cần thiết. Khi một component được bọc bởi React.memo, React sẽ so sánh props mới với props cũ. Nếu props không thay đổi, React sẽ bỏ qua việc render lại component đó và dùng lại kết quả render trước đó. Nhờ vậy, React.memo giúp giảm số lần render và cải thiện hiệu năng, đặc biệt khi component con nhận props giống nhau nhiều lần.

[EN] In React, React.memo is a performance optimization used for function components to prevent unnecessary re-renders. When a component is wrapped with React.memo, React will compare the new props with the previous props. If the props have not changed, React will skip re-rendering the component and reuse the previous render result. This helps

reduce the number of renders and improve performance, especially when a child component receives the same props many times.

[VN] React.memo khác useMemo ở điểm nào?

[EN] What is the difference between React.memo and useMemo?

[VN] Trong React, React.memo và useMemo đều dùng để tối ưu hiệu năng nhưng chúng hoạt động ở mức khác nhau. React.memo được dùng để ghi nhớ kết quả render của một component, và React sẽ bỏ qua việc render lại nếu props không thay đổi. Trong khi đó, useMemo được dùng bên trong component để ghi nhớ một giá trị hoặc kết quả của phép tính, giúp tránh tính toán lại khi component render nếu dependency không thay đổi. Nói đơn giản, React.memo tối ưu việc render component, còn useMemo tối ưu việc tính toán giá trị.

[EN] In React, React.memo and useMemo are both used for performance optimization but they work at different levels. React.memo is used to memoize the render result of a component, and React will skip re-rendering the component if the props have not changed. On the other hand, useMemo is used inside a component to memoize a value or the result of a calculation, so React does not recompute it when the component re-renders if the dependencies stay the same. In simple terms, React.memo optimizes component rendering, while useMemo optimizes value computation.

[VN] Khi nào nên dùng React.memo cho component?

[EN] When should you use React.memo for a component?

[VN] Trong React, nên dùng React.memo khi một component thường xuyên bị render lại nhưng props của nó ít khi thay đổi. Trong trường hợp này, React.memo sẽ giúp React so sánh props cũ và mới, và nếu props không thay đổi thì React sẽ bỏ qua việc render lại component đó. Điều này giúp giảm số lần render không cần thiết và cải thiện hiệu năng, đặc biệt khi component có logic phức tạp hoặc render tốn nhiều tài nguyên.

[EN] In React, you should use React.memo when a component re-renders frequently but its props rarely change. In this case, React.memo allows React to compare the previous and new props, and if they are the same, React will skip re-rendering that component. This helps reduce unnecessary renders and improve performance, especially when the component has complex logic or expensive rendering.

[VN] Inline function hoặc object trong props gây re-render như thế nào?

[EN] How do inline functions or objects in props cause re-render?

[VN] Trong React, khi truyền inline function hoặc object trong props, mỗi lần component cha render lại thì một function hoặc object mới sẽ được tạo ra. Mặc dù nội dung có thể

giống nhau, nhưng vì reference của chúng khác nhau nên React sẽ coi props đã thay đổi. Điều này khiến component con bị render lại, đặc biệt khi dùng `React.memo` vì React chỉ so sánh props theo reference. Vì vậy trong một số trường hợp người ta dùng `useCallback` hoặc `useMemo` để giữ reference ổn định và tránh re-render không cần thiết.

[EN] In React, when passing inline functions or objects as props, a new function or object is created every time the parent component re-renders. Even if the content is the same, the reference is different, so React considers the props as changed. This causes the child component to re-render, especially when using `React.memo` because React compares props by reference. In some cases, developers use `useCallback` or `useMemo` to keep the reference stable and avoid unnecessary re-renders.

[VN] Lifting state up là gì?

[EN] What is lifting state up?

[VN] Trong React, lifting state up là kỹ thuật di chuyển state từ component con lên component cha để nhiều component có thể chia sẻ và sử dụng cùng một dữ liệu. Thay vì mỗi component con tự quản lý state riêng, state sẽ được đặt ở component cha và truyền xuống các component con thông qua props. Cách này giúp dữ liệu được đồng bộ và dễ quản lý hơn, đặc biệt khi nhiều component cần cùng một trạng thái.

[EN] In React, lifting state up is a technique where state is moved from a child component to a parent component so that multiple components can share and use the same data. Instead of each child component managing its own state, the state is placed in the parent component and passed down to child components through props. This approach helps keep the data synchronized and easier to manage, especially when multiple components need the same state.

[VN] Props drilling là gì và tại sao nó gây khó chịu?

[EN] What is props drilling and why is it annoying?

[VN] Trong React, props drilling là tình huống khi một giá trị props phải được truyền qua nhiều component trung gian chỉ để đến được component con cần sử dụng nó. Những component trung gian này thực ra không dùng dữ liệu đó, nhưng vẫn phải nhận và truyền tiếp xuống. Điều này làm code dài hơn, khó đọc và khó bảo trì, đặc biệt khi cây component lớn. Vì vậy props drilling thường gây khó chịu cho lập trình viên.

[EN] In React, props drilling happens when a prop needs to be passed through multiple intermediate components just to reach the child component that actually needs it. These intermediate components do not use the data, but they still have to receive and pass it down. This makes the code longer, harder to read, and harder to maintain, especially when

the component tree becomes large. That is why props drilling is often frustrating for developers.

[VN] Khi nào nên dùng Context API thay vì chỉ truyền props?

[EN] When should you use Context API instead of just passing props?

[VN] Trong React, nên dùng Context API khi một dữ liệu cần được chia sẻ cho nhiều component ở nhiều cấp khác nhau trong cây component, ví dụ như theme, thông tin người dùng hoặc ngôn ngữ. Nếu chỉ truyền props qua nhiều component trung gian (props drilling) thì code sẽ khó đọc và khó bảo trì. Context API cho phép component con truy cập trực tiếp dữ liệu từ context mà không cần truyền props qua từng cấp.

[EN] In React, you should use the Context API when some data needs to be shared by many components at different levels in the component tree, such as theme, user information, or language settings. If you pass props through many intermediate components (props drilling), the code can become hard to read and maintain. The Context API allows child components to access the shared data directly from the context without passing props through every level.

[VN] useRef khác useState ở những điểm nào?

[EN] What are the differences between useRef and useState?

[VN] Trong React, useRef và useState khác nhau chủ yếu ở cách chúng ảnh hưởng đến việc render của component. useState dùng để lưu state và khi state thay đổi thì component sẽ re-render để cập nhật giao diện. Trong khi đó, useRef dùng để lưu một giá trị có thể thay đổi nhưng không làm component re-render khi giá trị đó thay đổi. useRef thường được dùng để truy cập trực tiếp DOM hoặc lưu giá trị giữa các lần render, còn useState thường dùng để quản lý dữ liệu ảnh hưởng đến UI.

[EN] In React, the main difference between useRef and useState is how they affect component rendering. useState is used to store state, and when the state changes, the component re-renders to update the UI. On the other hand, useRef is used to store a mutable value that does not cause a re-render when it changes. useRef is commonly used to access the DOM directly or keep values between renders, while useState is usually used to manage data that affects the UI.

[VN] Khi nào nên dùng useRef thay vì useState để tránh re-render?

[EN] When should you use useRef instead of useState to avoid re-render?

[VN] Trong React, nên dùng useRef thay vì useState khi cần lưu một giá trị thay đổi nhưng không ảnh hưởng đến giao diện và không muốn component bị re-render. Ví dụ như lưu ID của timer, lưu giá trị trước đó, hoặc tham chiếu đến DOM element. Khi giá trị trong useRef

thay đổi, React sẽ không render lại component, vì vậy nó giúp tránh các lần render không cần thiết.

[EN] In React, you should use `useRef` instead of `useState` when you need to store a value that changes but does not affect the UI and you want to avoid re-rendering the component. For example, it is commonly used to store a timer ID, keep a previous value, or reference a DOM element. When the value inside `useRef` changes, React does not trigger a re-render, which helps avoid unnecessary renders.

[VN] Batch update (automatic batching) trong React là gì?

[EN] What is batch update (automatic batching) in React?

[VN] Trong React, automatic batching là cơ chế mà React gom nhiều lần cập nhật state lại và xử lý chúng trong một lần render duy nhất. Điều này giúp giảm số lần render của component và cải thiện hiệu năng. Ví dụ, nếu nhiều `setState` được gọi gần nhau trong cùng một sự kiện hoặc logic bất đồng bộ, React sẽ batch các cập nhật đó lại và chỉ render một lần sau khi tất cả cập nhật hoàn thành.

[EN] In React, automatic batching is a mechanism where React groups multiple state updates together and processes them in a single render. This helps reduce the number of component renders and improves performance. For example, if multiple `setState` calls happen close together in the same event or asynchronous logic, React will batch those updates and perform only one render after all updates are processed.

[VN] React batch state updates khác nhau giữa event handler và async code như thế nào?

[EN] How do React batch state updates differ between event handlers and async code?

[VN] Trong React, batch state updates có sự khác nhau giữa event handler và async code (đặc biệt trong các phiên bản React cũ). Trong event handler như `onClick`, React sẽ tự động gom nhiều lần `setState` lại và chỉ render một lần. Tuy nhiên trong async code như `setTimeout`, `Promise` hoặc gọi API, trước đây React không tự batch, nên mỗi `setState` có thể gây ra một lần render riêng. Từ React 18 trở đi, React đã hỗ trợ automatic batching cho cả async code, giúp giảm số lần render và cải thiện hiệu năng.

[EN] In React, batch state updates behave differently between event handlers and async code (especially in older React versions). In event handlers like `onClick`, React automatically groups multiple `setState` calls and performs only one render. However, in async code such as `setTimeout`, `Promise`, or API calls, React previously did not batch updates, so each `setState` could trigger a separate render. Starting from React 18, React

introduced automatic batching for async code as well, which reduces the number of renders and improves performance.

[VN] Concurrent Rendering trong React 18 là gì?

[EN] What is Concurrent Rendering in React 18?

[VN] Trong React phiên bản 18, Concurrent Rendering là cơ chế cho phép React chuẩn bị việc render theo cách linh hoạt và có thể tạm dừng, tiếp tục hoặc ưu tiên các tác vụ khác. Điều này giúp React xử lý các cập nhật giao diện mượt hơn, đặc biệt khi có nhiều tác vụ nặng xảy ra cùng lúc. Ví dụ, React có thể ưu tiên các tương tác quan trọng của người dùng trước, trong khi các cập nhật ít quan trọng hơn có thể được xử lý sau.

[EN] In React version 18, Concurrent Rendering is a mechanism that allows React to prepare rendering in a flexible way, where it can pause, resume, or prioritize different tasks. This helps React keep the UI responsive, especially when multiple heavy updates happen at the same time. For example, React can prioritize important user interactions first, while less important updates can be processed later.

[VN] useTransition dùng để làm gì?

[EN] What is useTransition used for?

[VN] Trong React, useTransition là một hook dùng để đánh dấu một số cập nhật state là không khẩn cấp (non-urgent). Điều này cho phép React ưu tiên các tương tác quan trọng của người dùng, như nhập dữ liệu hoặc click, trước khi xử lý các cập nhật nặng hơn. Khi dùng useTransition, React có thể giữ giao diện phản hồi nhanh trong khi các cập nhật ít quan trọng hơn được xử lý ở nền.

[EN] In React, useTransition is a hook used to mark some state updates as non-urgent. This allows React to prioritize important user interactions, such as typing or clicking, before processing heavier updates. With useTransition, React can keep the UI responsive while less important updates are handled in the background.

[VN] useTransition giúp cải thiện UX trong trường hợp nào?

[EN] In what situations does useTransition improve UX?

[VN] Trong React, useTransition giúp cải thiện trải nghiệm người dùng (UX) khi có các cập nhật giao diện nặng hoặc tốn thời gian, ví dụ như lọc danh sách lớn, tìm kiếm dữ liệu, hoặc render nhiều component cùng lúc. Trong những trường hợp này, useTransition cho phép React ưu tiên các tương tác quan trọng như nhập dữ liệu hoặc click, trong khi các cập nhật ít quan trọng hơn được xử lý sau. Nhờ vậy giao diện vẫn mượt và phản hồi nhanh, thay vì bị chậm hoặc bị “đơ”.

[EN] In React, `useTransition` improves user experience (UX) when the UI has heavy or time-consuming updates, such as filtering a large list, searching data, or rendering many components at once. In these situations, `useTransition` allows React to prioritize important user interactions like typing or clicking, while less important updates are processed later. This keeps the interface smooth and responsive instead of feeling slow or frozen.

[VN] StrictMode trong React dùng để làm gì?

[EN] What is StrictMode used for in React?

[VN] StrictMode trong React là một công cụ dành cho môi trường phát triển (development) giúp lập trình viên phát hiện các vấn đề tiềm ẩn trong ứng dụng. Khi bật StrictMode, React sẽ kiểm tra và cảnh báo về các cách dùng không an toàn, API đã lỗi thời, hoặc các side effects có thể gây lỗi trong tương lai. Nó cũng có thể chạy một số hàm hai lần trong development để giúp phát hiện bug liên quan đến state hoặc side effects. StrictMode không ảnh hưởng đến code khi chạy ở môi trường production.

[EN] StrictMode in React is a development tool that helps developers find potential problems in an application. When StrictMode is enabled, React checks and warns about unsafe practices, deprecated APIs, or side effects that may cause issues in the future. It may also run some functions twice in development to help detect bugs related to state or side effects. StrictMode does not affect the behavior of the application in production.

[VN] Tại sao useEffect chạy 2 lần trong StrictMode (ở development)?

[EN] Why does useEffect run twice in StrictMode (in development)?

[VN] Trong React, `useEffect` có thể chạy 2 lần khi dùng StrictMode ở môi trường development vì React cố tình mount, unmount và mount lại component một lần nữa để kiểm tra các side effects có an toàn hay không. Cách này giúp phát hiện các lỗi như side effects không được cleanup đúng cách, gọi API nhiều lần không mong muốn, hoặc logic phụ thuộc vào việc component chỉ chạy một lần. Điều này chỉ xảy ra trong development để giúp lập trình viên phát hiện bug sớm, và sẽ không xảy ra trong môi trường production.

[EN] In React, `useEffect` may run twice when using StrictMode in development because React intentionally mounts, unmounts, and mounts the component again to check if side effects are safe. This helps detect problems such as missing cleanup, unintended multiple API calls, or logic that incorrectly assumes the component runs only once. This behavior only happens in development to help developers find bugs early and does not occur in production.

[VN] Suspense component hoạt động như thế nào?

[EN] How does the Suspense component work?

[VN] Trong React, Suspense là một component dùng để xử lý các tác vụ bất đồng bộ (như lazy loading component hoặc tải dữ liệu). Khi một component con chưa sẵn sàng (ví dụ đang tải), Suspense sẽ tạm thời hiển thị nội dung dự phòng được truyền qua prop fallback (ví dụ: loading spinner). Khi dữ liệu hoặc component đã tải xong, React sẽ tự động render lại và hiển thị nội dung thật. Cách này giúp giao diện không bị đứng và mang lại trải nghiệm người dùng mượt hơn.

[EN] In React, Suspense is a component used to handle asynchronous tasks such as lazy loading components or fetching data. When a child component is not ready yet (for example, still loading), Suspense temporarily shows a fallback UI provided through the fallback prop (such as a loading spinner). Once the data or component is ready, React automatically re-renders and displays the real content. This helps keep the UI responsive and improves user experience.

[VN] Suspense có thể dùng cho data fetching không?

[EN] Can Suspense be used for data fetching?

[VN] Trong React, Suspense có thể dùng cho data fetching, nhưng nó không hoạt động trực tiếp với mọi cách gọi API thông thường. Suspense hoạt động khi một component “tạm dừng” việc render cho đến khi dữ liệu sẵn sàng, và trong thời gian đó React sẽ hiển thị UI dự phòng thông qua fallback. Tuy nhiên, để dùng Suspense cho data fetching, chúng ta thường cần các thư viện hoặc framework hỗ trợ như React Query, Relay, hoặc Next.js, vì chúng được thiết kế để tích hợp tốt với cơ chế Suspense.

[EN] In React, Suspense can be used for data fetching, but it does not work directly with normal API calls by default. Suspense works by letting a component “pause” rendering until the data is ready, and during that time React shows a fallback UI through the fallback prop. However, to use Suspense for data fetching, developers usually use libraries or frameworks that support it, such as React Query, Relay, or Next.js, because they are designed to integrate well with Suspense.

[VN] Error Boundary là gì và cách hoạt động?

[EN] What is an Error Boundary and how does it work?

[VN] Trong React, Error Boundary là một component đặc biệt dùng để bắt lỗi JavaScript xảy ra trong quá trình render, lifecycle hoặc constructor của các component con, giúp ứng dụng không bị crash toàn bộ. Khi một lỗi xảy ra, Error Boundary sẽ chặn lỗi đó, ghi log nếu cần và hiển thị một UI dự phòng (fallback UI) thay vì để ứng dụng bị dừng. Nó thường được tạo bằng class component với các method như componentDidCatch hoặc getDerivedStateFromError, và được đặt bao quanh các component có khả năng xảy ra lỗi để tăng độ ổn định cho ứng dụng.

[EN] In React, an Error Boundary is a special component used to catch JavaScript errors that happen during rendering, lifecycle methods, or constructors of child components, so the whole application does not crash. When an error occurs, the Error Boundary catches it, can log the error if needed, and shows a fallback UI instead of breaking the entire app. It is usually implemented using a class component with methods like `componentDidCatch` or `getDerivedStateFromError`, and it is placed around components that may fail to make the application more stable.

[VN] Code splitting trong React làm như thế nào?

[EN] How does code splitting work in React?

[VN] Trong React, code splitting là kỹ thuật chia nhỏ bundle JavaScript thành nhiều phần để chỉ tải khi cần, giúp giảm thời gian tải trang ban đầu. Cách phổ biến là dùng `React.lazy()` để load component động và bọc nó bằng `Suspense` để hiển thị UI tạm thời (fallback) trong lúc component đang được tải. Khi người dùng truy cập vào phần cần component đó, React mới tải file tương ứng thay vì tải toàn bộ ứng dụng ngay từ đầu.

[EN] In React, code splitting is a technique that splits a large JavaScript bundle into smaller pieces so they are loaded only when needed, which reduces the initial page load time. A common way is to use `React.lazy()` to load components dynamically and wrap them with `Suspense` to show a fallback UI while the component is loading. This means the component's code is downloaded only when the user navigates to the part of the app that needs it, instead of loading the entire application at once.

[VN] React.lazy và Suspense dùng để lazy-load component ra sao?

[EN] How do React.lazy and Suspense lazy-load components?

[VN] Trong React, `React.lazy` và `Suspense` được dùng để lazy-load component, nghĩa là chỉ tải component khi thật sự cần. `React.lazy()` cho phép import component một cách động (dynamic import), nên file JavaScript của component đó chỉ được tải khi nó được render. `Suspense` được dùng để bao quanh component lazy và hiển thị một giao diện tạm thời (fallback), ví dụ như “Loading...”, trong lúc component đang được tải. Khi file component tải xong, React sẽ render component thật thay cho fallback.

[EN] In React, `React.lazy` and `Suspense` are used to lazy-load components, meaning the component code is loaded only when it is needed. `React.lazy()` allows dynamic import of a component, so its JavaScript file is downloaded only when the component is rendered. `Suspense` wraps the lazy component and shows a temporary UI (fallback), such as “Loading...”, while the component is being loaded. Once the file finishes loading, React renders the real component instead of the fallback.

[VN] Portals trong React là gì?

[EN] What are Portals in React?

[VN] Trong React, Portals là cách cho phép một component render nội dung của nó vào một vị trí khác trong DOM, bên ngoài cây DOM chính của component cha. Điều này thường được dùng cho các UI như modal, tooltip hoặc popup, nơi mà phần tử cần hiển thị ở một vị trí khác trong HTML để tránh vấn đề về CSS hoặc layout. Dù được render ở vị trí khác trong DOM, component vẫn thuộc cùng cây React nên vẫn có thể sử dụng props, state và event như bình thường.

[EN] In React, Portals allow a component to render its content into a different place in the DOM, outside the main DOM tree of its parent component. This is commonly used for UI elements like modals, tooltips, or popups, where the element needs to appear in another place in the HTML to avoid CSS or layout issues. Even though it is rendered in a different DOM location, the component is still part of the same React tree, so it can still use props, state, and events normally.

[VN] Dùng Portal trong những trường hợp nào (ví dụ cụ thể)?

[EN] In what situations are Portals used (specific examples)?

[VN] Trong React, Portal thường được dùng khi một component cần hiển thị bên ngoài cấu trúc DOM của component cha nhưng vẫn giữ logic trong cây React. Ví dụ phổ biến là modal dialog, tooltip, dropdown menu, hoặc notification popup. Những thành phần này thường cần hiển thị trên cùng của trang (overlay) và không bị ảnh hưởng bởi CSS như overflow, z-index, hoặc layout của component cha. Vì vậy, Portal cho phép render chúng vào một node khác trong DOM (ví dụ document.body) để đảm bảo hiển thị đúng và dễ quản lý.

[EN] In React, Portal is used when a component needs to render outside the DOM structure of its parent component but still stay in the same React tree. Common examples include modal dialogs, tooltips, dropdown menus, and notification popups. These UI elements often need to appear on top of the page and should not be affected by CSS rules like overflow, z-index, or layout from the parent component. Portals allow them to render into another DOM node (such as document.body) so they display correctly and remain easy to manage.

[VN] Tại sao Hooks không thể bắt error như Error Boundary trong class?

[EN] Why can't Hooks catch errors like Error Boundaries in class components?

[VN] Trong React, Hooks không thể bắt lỗi giống Error Boundary vì Hooks chỉ chạy trong function component và dùng để quản lý state hoặc side effects. Khi một lỗi xảy ra trong

quá trình render, lifecycle, hoặc constructor của component con, React cần một cơ chế đặc biệt để chặn lỗi và hiển thị UI dự phòng (fallback UI). Cơ chế này chỉ được hỗ trợ trong class component thông qua các method như componentDidCatch và getDerivedStateFromError. Hooks không có quyền can thiệp vào quá trình render ở mức đó, nên chúng không thể hoạt động như Error Boundary.

[EN] In React, Hooks cannot catch errors like Error Boundaries because Hooks only run inside function components and are designed to manage state and side effects. When an error happens during rendering, lifecycle, or constructor of child components, React needs a special mechanism to stop the error and show a fallback UI. This mechanism is only supported in class components through methods like componentDidCatch and getDerivedStateFromError. Hooks do not have control over the rendering process at that level, so they cannot work as an Error Boundary.

[VN] Higher-Order Component (HOC) là gì?

[EN] What is a Higher-Order Component (HOC)?

[VN] Trong React, Higher-Order Component (HOC) là một pattern trong đó một function nhận vào một component và trả về một component mới. Component mới này sẽ bao bọc component gốc và có thể thêm logic chung như kiểm tra quyền truy cập, xử lý dữ liệu, hoặc logging. Mục đích của HOC là tái sử dụng logic giữa nhiều component mà không cần lặp lại code. Ví dụ, một HOC có thể thêm chức năng kiểm tra người dùng đã đăng nhập trước khi cho phép component hiển thị.

[EN] In React, a Higher-Order Component (HOC) is a pattern where a function takes a component as input and returns a new component. The new component wraps the original component and can add shared logic such as authentication checks, data handling, or logging. The main goal of HOC is to reuse logic across multiple components without repeating code. For example, a HOC can add a feature that checks if a user is logged in before allowing the component to render.

[VN] Render Props pattern là gì?

[EN] What is the Render Props pattern?

[VN] Trong React, Render Props là một pattern trong đó một component nhận vào một prop là một function, và component đó sẽ gọi function này để quyết định nội dung cần render. Function này thường nhận dữ liệu từ component cha và trả về JSX để hiển thị. Mục đích của Render Props là chia sẻ logic giữa nhiều component mà không cần lặp lại code, ví dụ một component có thể xử lý logic lấy dữ liệu hoặc theo dõi trạng thái chuột, rồi truyền dữ liệu đó cho function để render UI khác nhau.

[EN] In React, Render Props is a pattern where a component receives a prop that is a function, and the component calls this function to decide what UI to render. This function usually receives data from the component and returns JSX to display. The goal of Render Props is to share logic between multiple components without repeating code. For example, a component can handle logic like fetching data or tracking mouse position, then pass that data to the function so different UIs can be rendered.

[VN] So sánh HOC vs Render Props vs Custom Hooks?

[EN] Compare HOC vs Render Props vs Custom Hooks?

[VN] Trong React, HOC (Higher-Order Component), Render Props, và Custom Hooks đều là các cách để tái sử dụng logic giữa nhiều component. HOC là một function nhận vào một component và trả về một component mới có thêm logic. Render Props là pattern trong đó một component nhận một prop là function và gọi function đó để render UI dựa trên dữ liệu bên trong. Custom Hooks là cách hiện đại hơn, cho phép tách logic vào một hook riêng và sử dụng lại trong nhiều function component một cách đơn giản. Hiện nay, Custom Hooks thường được ưu tiên hơn vì code dễ đọc, ít lồng component, và dễ bảo trì.

[EN] In React, HOC (Higher-Order Component), Render Props, and Custom Hooks are all ways to reuse logic across multiple components. HOC is a function that takes a component and returns a new component with additional logic. Render Props is a pattern where a component receives a function as a prop and calls that function to render UI using its internal data. Custom Hooks are a newer approach that lets developers extract logic into a reusable hook and use it in many function components easily. Today, Custom Hooks are usually preferred because the code is cleaner, has less component nesting, and is easier to maintain.

[VN] Ưu nhược điểm chính của Hooks so với HOC và Render Props?

[EN] What are the main advantages and disadvantages of Hooks compared to HOC and Render Props?

[VN] Trong React, Hooks được dùng để tái sử dụng logic giữa các component và thường được xem là cách hiện đại hơn so với HOC và Render Props. Ưu điểm của Hooks là code ngắn gọn hơn, dễ đọc, ít lồng component, và logic có thể tách ra thành custom hooks để dùng lại nhiều nơi. Tuy nhiên, Hooks cũng có nhược điểm như chỉ dùng được trong function component, phải tuân theo Rules of Hooks (không gọi trong vòng lặp hay điều kiện), và đôi khi việc quản lý dependency trong các hook như useEffect có thể gây bug nếu không cẩn thận.

[EN] In React, Hooks are used to reuse logic across components and are often considered a more modern approach compared to HOC and Render Props. The advantages of Hooks are

that the code is simpler, easier to read, and avoids deep component nesting, and logic can be extracted into custom hooks for reuse. However, Hooks also have some disadvantages: they only work in function components, must follow the Rules of Hooks (cannot be called inside loops or conditions), and managing dependencies in hooks like `useEffect` can sometimes cause bugs if not handled carefully.

[VN] Làm sao detect re-render không cần thiết trong React?

[EN] How to detect unnecessary re-renders in React?

[VN] Trong React, để phát hiện re-render không cần thiết, lập trình viên thường dùng các công cụ debug như React Developer Tools với tính năng Profiler để xem component nào render lại và mất bao nhiêu thời gian. Ngoài ra, có thể thêm `console.log` trong component để kiểm tra khi nào nó render. Một cách khác là dùng thư viện như `why-did-you-render`, thư viện này sẽ cảnh báo khi một component re-render dù props hoặc state không thay đổi.

[EN] In React, developers can detect unnecessary re-renders by using debugging tools such as React Developer Tools, especially the Profiler feature, which shows which components re-render and how long they take. Another simple way is to add `console.log` inside a component to see when it renders. Developers can also use libraries like `why-did-you-render`, which warns when a component re-renders even though its props or state have not changed.

[VN] Tại sao component re-render dù props không đổi?

[EN] Why does a component re-render even though props haven't changed?

[VN] Trong React, một component vẫn có thể re-render dù props không đổi vì nhiều lý do. Ví dụ, khi component cha re-render, các component con cũng sẽ render lại theo mặc định. Ngoài ra, nếu props là object, array hoặc function được tạo lại mỗi lần render (ví dụ inline function hoặc object mới), React sẽ coi đó là giá trị mới vì so sánh theo reference. Bên cạnh đó, khi state của chính component thay đổi hoặc context thay đổi, component cũng sẽ render lại dù props không thay đổi.

[EN] In React, a component can still re-render even if its props have not changed for several reasons. For example, when the parent component re-renders, child components also re-render by default. Also, if props are objects, arrays, or functions created again on every render (such as inline functions or new objects), React treats them as new values because it compares by reference. In addition, if the component's own state changes or the context value changes, the component will re-render even though its props remain the same.

[VN] Khi nào nên memoize (useMemo, useCallback, memo)?

[EN] When should you memoize (useMemo, useCallback, memo)?

[VN] Trong React, nên dùng memoization như useMemo, useCallback hoặc memo khi component hoặc phép tính tốn nhiều tài nguyên và bị render lại nhiều lần không cần thiết. Ví dụ, useMemo dùng để lưu kết quả của một phép tính nặng để tránh tính lại mỗi lần render, useCallback dùng để giữ cùng một reference của function khi truyền xuống component con, và memo giúp component con không render lại nếu props không thay đổi. Tuy nhiên, chỉ nên dùng khi thật sự cần tối ưu hiệu năng, vì nếu lạm dụng có thể làm code phức tạp hơn mà không mang lại lợi ích rõ ràng.

[EN] In React, memoization such as useMemo, useCallback, or memo should be used when a component or computation is expensive and may re-render many times unnecessarily. For example, useMemo stores the result of a heavy calculation so it is not recomputed on every render, useCallback keeps the same function reference when passing it to child components, and memo prevents a child component from re-rendering if its props have not changed. However, these optimizations should only be used when needed, because overusing them can make the code more complex without giving clear performance benefits.

[VN] Khi nào memoization trở thành over-optimization?

[EN] When does memoization become over-optimization?

[VN] Trong React, memoization trở thành over-optimization khi lập trình viên dùng useMemo, useCallback hoặc memo cho những phép tính đơn giản hoặc component rất nhỏ, nơi mà việc render lại gần như không tốn chi phí. Trong những trường hợp này, chi phí để React kiểm tra dependency hoặc so sánh props đôi khi còn tốn tài nguyên hơn việc render lại. Điều này làm code phức tạp hơn, khó đọc và khó bảo trì mà không mang lại lợi ích về hiệu năng.

[EN] In React, memoization becomes over-optimization when developers use useMemo, useCallback, or memo for very simple calculations or small components where re-rendering has almost no cost. In these cases, the overhead of checking dependencies or comparing props can sometimes cost more than simply re-rendering the component. This makes the code more complex and harder to maintain without providing real performance benefits.

[VN] Debounce và Throttle trong React dùng để làm gì?

[EN] What are Debounce and Throttle used for in React?

[VN] Trong React, Debounce và Throttle là các kỹ thuật dùng để giảm số lần một function được gọi khi có nhiều sự kiện xảy ra liên tục, giúp cải thiện hiệu năng. Debounce sẽ trì hoãn việc gọi function cho đến khi người dùng ngừng thực hiện hành động trong một

khoảng thời gian, ví dụ khi tìm kiếm trong ô input thì chỉ gửi request sau khi người dùng ngừng gõ. Throttle thì giới hạn việc gọi function chỉ xảy ra tối đa một lần trong một khoảng thời gian nhất định, ví dụ khi xử lý sự kiện scroll hoặc resize. Hai kỹ thuật này giúp tránh việc gọi API hoặc re-render quá nhiều lần không cần thiết.

[EN] In React, Debounce and Throttle are techniques used to limit how often a function is executed when many events happen quickly, which helps improve performance. Debounce delays the function call until the user stops performing an action for a short time, for example sending a search request only after the user stops typing in an input field. Throttle limits the function so it can run at most once within a certain time interval, which is useful for events like scrolling or resizing. Both techniques help prevent too many API calls or unnecessary re-renders.

[VN] Virtualization (windowing) là gì?

[EN] What is Virtualization (windowing)?

[VN] Trong React, Virtualization (hay còn gọi là windowing) là kỹ thuật chỉ render những phần tử đang hiển thị trên màn hình thay vì render toàn bộ danh sách lớn. Ví dụ, nếu có một danh sách hàng nghìn item, React chỉ render những item nằm trong vùng nhìn thấy của người dùng và sẽ cập nhật khi người dùng scroll. Cách này giúp giảm số lượng DOM elements, cải thiện hiệu năng và làm ứng dụng chạy mượt hơn, đặc biệt khi làm việc với danh sách rất lớn.

[EN] In React, Virtualization (also called windowing) is a technique where only the items currently visible on the screen are rendered instead of rendering the entire large list. For example, if there are thousands of items in a list, React only renders the items within the visible area and updates them when the user scrolls. This approach reduces the number of DOM elements, improves performance, and makes the application smoother, especially when working with very large lists.

[VN] Khi nào cần dùng virtualization (ví dụ danh sách 10k items)?

[EN] When should virtualization be used (for example, a list of 10k items)?

[VN] Trong React, virtualization nên được sử dụng khi ứng dụng cần hiển thị danh sách rất lớn, ví dụ hàng nghìn hoặc hàng chục nghìn items (như 10,000 items). Nếu render toàn bộ danh sách cùng lúc, trình duyệt phải tạo rất nhiều DOM elements, điều này có thể làm ứng dụng chậm, lag hoặc tốn nhiều bộ nhớ. Virtualization giúp chỉ render các item đang hiển thị trên màn hình và cập nhật khi người dùng scroll, vì vậy hiệu năng sẽ tốt hơn và trải nghiệm người dùng mượt hơn.

[EN] In React, virtualization should be used when an application needs to display very large lists, such as thousands or tens of thousands of items (for example, 10,000 items). If the entire list is rendered at once, the browser has to create many DOM elements, which can make the application slow, laggy, and use more memory. Virtualization solves this by rendering only the items visible on the screen and updating them when the user scrolls, which improves performance and provides a smoother user experience.

[VN] Cách phổ biến để tối ưu bundle size trong React app?

[EN] Common ways to optimize bundle size in a React app?

[VN] Trong React, để tối ưu bundle size, lập trình viên thường dùng các kỹ thuật như code splitting để chia code thành nhiều phần nhỏ và chỉ tải khi cần. Một cách phổ biến là sử dụng React.lazy kết hợp với Suspense để lazy-load component. Ngoài ra, có thể giảm bundle size bằng cách loại bỏ thư viện không cần thiết, dùng tree shaking, và nén code bằng các công cụ build. Những cách này giúp ứng dụng tải nhanh hơn và cải thiện trải nghiệm người dùng.

[EN] In React, developers often optimize bundle size using techniques such as code splitting, which divides the code into smaller chunks that are loaded only when needed. A common method is using React.lazy together with Suspense to lazy-load components. Developers can also reduce bundle size by removing unnecessary libraries, using tree shaking, and minifying code with build tools. These methods help the application load faster and improve the user experience.

[VN] Redux hoạt động theo ba nguyên lý chính nào?

[EN] What are the three core principles of Redux?

[VN] Trong Redux, có ba nguyên lý chính. Thứ nhất là Single source of truth, nghĩa là toàn bộ state của ứng dụng được lưu trong một store duy nhất. Thứ hai là State là read-only, nghĩa là state không được thay đổi trực tiếp mà chỉ có thể thay đổi bằng cách gửi action. Thứ ba là Changes are made with pure functions, nghĩa là state mới được tạo ra bằng reducer, một hàm thuần (pure function) nhận state cũ và action để trả về state mới. Ba nguyên lý này giúp state dễ quản lý, dễ dự đoán và dễ debug.

[EN] In Redux, there are three main principles. The first is Single source of truth, meaning the entire application state is stored in a single store. The second is State is read-only, meaning the state cannot be changed directly and can only be updated by sending an action. The third is Changes are made with pure functions, meaning a new state is created by a reducer, which is a pure function that takes the previous state and an action to return the new state. These principles make the state easier to manage, predictable, and easier to debug.

[VN] Flow dữ liệu cơ bản trong Redux: dispatch → reducer → store?

[EN] Basic data flow in Redux: dispatch → reducer → store?

[VN] Trong Redux, flow dữ liệu cơ bản diễn ra theo các bước dispatch → reducer → store. Đầu tiên, component gửi một action bằng cách gọi dispatch. Sau đó reducer nhận action này cùng với state hiện tại và tạo ra state mới dựa trên logic đã định nghĩa. Cuối cùng, store được cập nhật với state mới và các component đang subscribe sẽ render lại để hiển thị dữ liệu mới. Flow này luôn đi theo một chiều, giúp dữ liệu dễ theo dõi và dự đoán.

[EN] In Redux, the basic data flow follows dispatch → reducer → store. First, a component sends an action by calling dispatch. Then the reducer receives this action along with the current state and returns a new state based on the defined logic. Finally, the store is updated with the new state, and components that subscribe to the store will re-render to display the updated data. This flow always moves in one direction, making the data easier to track and predict.

[VN] Tại sao reducer phải là pure function?

[EN] Why must a reducer be a pure function?

[VN] Trong Redux, reducer phải là pure function vì nó giúp việc quản lý state dễ dự đoán và dễ debug. Pure function nghĩa là với cùng input (state và action) thì reducer luôn trả về cùng một kết quả, và nó không thay đổi state cũ, không gọi API, không tạo side effects. Nhờ vậy, Redux có thể dễ dàng theo dõi sự thay đổi của state, hỗ trợ các tính năng như time-travel debugging và giúp ứng dụng hoạt động ổn định hơn.

[EN] In Redux, a reducer must be a pure function because it makes state management predictable and easier to debug. A pure function means that with the same input (state and action) it always returns the same result, and it does not modify the old state, call APIs, or cause side effects. Because of this, Redux can easily track state changes, support features like time-travel debugging, and make the application more stable.

[VN] Immutability quan trọng như thế nào trong Redux?

[EN] How important is immutability in Redux?

[VN] Trong Redux, immutability (tính bất biến) rất quan trọng vì state không được thay đổi trực tiếp, mà phải tạo ra một state mới mỗi khi có thay đổi. Điều này giúp Redux dễ dàng so sánh state cũ và state mới, từ đó phát hiện thay đổi và cập nhật UI chính xác. Ngoài ra, immutability còn giúp việc debug, theo dõi lịch sử state và sử dụng time-travel debugging trở nên dễ dàng hơn.

[EN] In Redux, immutability is very important because the state should not be modified directly, and a new state object must be created whenever changes happen. This allows Redux to easily compare the previous state and the new state to detect updates and refresh the UI correctly. Immutability also makes debugging, tracking state history, and using time-travel debugging much easier.

[VN] Single source of truth trong Redux mang lại lợi ích gì?

[EN] What benefits does single source of truth provide in Redux?

[VN] Trong Redux, Single source of truth nghĩa là toàn bộ state của ứng dụng được lưu trong một store duy nhất. Điều này giúp dữ liệu nhất quán, dễ quản lý và dễ theo dõi vì mọi thay đổi đều đi qua cùng một nơi. Nhờ vậy, lập trình viên có thể debug dễ hơn, theo dõi lịch sử thay đổi của state, và tránh tình trạng dữ liệu bị lệch giữa nhiều component.

[EN] In Redux, Single source of truth means that the entire application state is stored in one single store. This makes the data consistent, easier to manage, and easier to track, because all changes go through the same place. As a result, developers can debug more easily, track state history, and avoid situations where data becomes inconsistent across multiple components.

[VN] Tại sao không nên lưu derived state (state tính toán được) trong store?

[EN] Why should derived state (computed state) not be stored in the store?

[VN] Trong Redux, không nên lưu derived state (state có thể tính toán từ state khác) trong store vì nó có thể gây trùng lặp dữ liệu và khó đồng bộ. Nếu dữ liệu gốc thay đổi nhưng derived state không được cập nhật đúng cách, ứng dụng có thể hiển thị dữ liệu sai hoặc không nhất quán. Thay vào đó, nên lưu state gốc (source data) trong store và tính toán derived state khi cần, thường bằng selector.

[EN] In Redux, it is not recommended to store derived state (state that can be calculated from other state) in the store because it can cause duplicate data and synchronization problems. If the original data changes but the derived state is not updated correctly, the application may show incorrect or inconsistent data. Instead, developers should store the source data in the store and calculate the derived state when needed, usually using selectors.

[VN] Redux khác Context API ở những điểm chính nào?

[EN] What are the main differences between Redux and Context API?

[VN] Trong Redux và React Context API, cả hai đều dùng để chia sẻ state giữa nhiều component, nhưng mục đích và cách sử dụng khác nhau. Redux là một thư viện quản lý state mạnh mẽ với cấu trúc rõ ràng như store, action, reducer, phù hợp cho ứng dụng lớn và

phức tạp. Nó cũng có các công cụ hỗ trợ như middleware và devtools để debug. Trong khi đó, Context API là tính năng có sẵn trong React, đơn giản hơn và thường dùng để chia sẻ dữ liệu toàn cục nhỏ như theme, language hoặc user info. Tuy nhiên, nếu dùng Context cho state phức tạp hoặc thay đổi thường xuyên, có thể gây nhiều re-render và khó quản lý hơn.

[EN] Both Redux and the React Context API are used to share state between multiple components, but they serve different purposes and have different structures. Redux is a powerful state management library with a clear architecture such as store, action, and reducer, which makes it suitable for large and complex applications. It also provides tools like middleware and devtools for debugging. In contrast, Context API is a built-in feature of React, simpler and commonly used for sharing small global data such as theme, language, or user information. However, using Context for complex or frequently changing state may cause many re-renders and become harder to manage.

[VN] Khi nào nên dùng Redux, khi nào chỉ cần Context + useReducer?

[EN] When should Redux be used, and when is Context + useReducer enough?

[VN] Trong React, nên dùng Redux khi ứng dụng lớn, có nhiều state phức tạp, nhiều component cần chia sẻ dữ liệu và cần công cụ quản lý, debug tốt. Redux cung cấp cấu trúc rõ ràng, middleware và devtools để theo dõi state. Ngược lại, nếu ứng dụng nhỏ hoặc trung bình, state không quá phức tạp và chỉ cần chia sẻ dữ liệu giữa một số component, thì React Context API kết hợp với useReducer thường đơn giản hơn và đủ dùng mà không cần thêm thư viện bên ngoài.

[EN] In React, you should use Redux when the application is large, has complex state, many components need shared data, and better state management and debugging tools are required. Redux provides a clear structure, middleware, and devtools for tracking state changes. On the other hand, if the application is small or medium-sized, the state is not very complex, and only a few components need shared data, then React Context API with useReducer is usually simpler and sufficient without adding an external library.

[VN] Redux Thunk và Redux Saga khác nhau cơ bản như thế nào?

[EN] What are the fundamental differences between Redux Thunk and Redux Saga?

[VN] Trong Redux, Redux Thunk và Redux-Saga đều là middleware dùng để xử lý logic bất đồng bộ (async) như gọi API. Redux Thunk cho phép action trả về một function thay vì object, và function này có thể thực hiện async rồi dispatch action khác; cách này đơn giản và dễ học. Trong khi đó, Redux Saga sử dụng generator function để quản lý các luồng async phức tạp, giúp code rõ ràng hơn khi có nhiều bước như retry, delay hoặc xử lý song song, nhưng cũng phức tạp và khó học hơn.

[EN] In Redux, Redux Thunk and Redux-Saga are middleware used to handle asynchronous logic such as API calls. Redux Thunk allows an action to return a function instead of an object, and this function can perform async tasks and then dispatch other actions; it is simple and easy to learn. In contrast, Redux Saga uses generator functions to manage complex async flows, making code clearer when handling cases like retry, delay, or parallel tasks, but it is more complex and harder to learn.

[VN] Backend-for-Frontend (BFF) pattern dùng khi nào?

[EN] When should the Backend-for-Frontend (BFF) pattern be used?

[VN] Backend-for-Frontend (BFF) pattern được dùng khi hệ thống có nhiều loại client khác nhau như web, mobile hoặc IoT và mỗi client cần dữ liệu hoặc API khác nhau. Thay vì tất cả client gọi chung một backend, mỗi loại client sẽ có một backend riêng được tối ưu cho nhu cầu của nó. BFF có thể tổng hợp dữ liệu từ nhiều microservice, chuyển đổi format dữ liệu và giảm số lượng request từ client. Cách này giúp tăng hiệu năng, đơn giản hóa client và dễ thay đổi logic cho từng loại giao diện.

[EN] The Backend-for-Frontend (BFF) pattern is used when a system has multiple types of clients, such as web, mobile, or IoT, and each client needs different APIs or data. Instead of all clients calling the same backend, each client has its own backend optimized for its needs. The BFF can aggregate data from multiple microservices, transform data formats, and reduce the number of requests from the client. This approach improves performance, simplifies the client, and allows easier changes for each user interface.

X. Frontend – Reacts

Virtual DOM trong React là gì?

Cơ chế diffing (reconciliation) của React hoạt động như thế nào?

Sự khác biệt chính giữa Virtual DOM và DOM thật là gì?

Tại sao key lại quan trọng khi render danh sách trong React?

Nếu dùng index làm key trong list, sẽ gặp những vấn đề gì?

Ví dụ cụ thể về trường hợp key bị sai dẫn đến bug giao diện?

Controlled component trong React là gì?

Uncontrolled component là gì và cách sử dụng?

Khi nào nên dùng controlled component, khi nào nên dùng uncontrolled?

SyntheticEvent trong React là gì?

SyntheticEvent khác native DOM event ở những điểm nào?

Tại sao React cần dùng SyntheticEvent thay vì event native?

useEffect trong React hoạt động như thế nào?

Dependency array trong useEffect có vai trò gì?

Khi dependency array là [] thì useEffect chạy lúc nào?
Khi không truyền dependency array thì useEffect chạy ra sao?
Cleanup function trong useEffect dùng để làm gì?
Cho ví dụ thực tế về cleanup trong useEffect (subscription, timer...).
Tại sao cleanup chạy trước khi component unmount và trước khi effect chạy lại?
useMemo dùng để làm gì?
useCallback dùng để làm gì?
Điểm khác biệt chính giữa useMemo và useCallback?
Khi nào KHÔNG nên dùng useMemo hoặc useCallback?
React.memo là gì và hoạt động như thế nào?
React.memo khác useMemo ở điểm nào?
Khi nào nên dùng React.memo cho component?
Inline function hoặc object trong props gây re-render như thế nào?
Lifting state up là gì?
Props drilling là gì và tại sao nó gây khó chịu?
Khi nào nên dùng Context API thay vì chỉ truyền props?
useRef khác useState ở những điểm nào?
Khi nào nên dùng useRef thay vì useState để tránh re-render?
Batch update (automatic batching) trong React là gì?
React batch state updates khác nhau giữa event handler và async code như thế nào?
Concurrent Rendering trong React 18 là gì?
useTransition dùng để làm gì?
useTransition giúp cải thiện UX trong trường hợp nào?
StrictMode trong React dùng để làm gì?
Tại sao useEffect chạy 2 lần trong StrictMode (ở development)?
Suspense component hoạt động như thế nào?
Suspense có thể dùng cho data fetching không?
Error Boundary là gì và cách hoạt động?
Code splitting trong React làm như thế nào?
React.lazy và Suspense dùng để lazy-load component ra sao?
Portals trong React là gì?
Dùng Portal trong những trường hợp nào (ví dụ cụ thể)?
Tại sao Hooks không thể bắt error như Error Boundary trong class?
Higher-Order Component (HOC) là gì?
Render Props pattern là gì?
So sánh HOC vs Render Props vs Custom Hooks?
Ưu nhược điểm chính của Hooks so với HOC và Render Props?
Làm sao detect re-render không cần thiết trong React?
Tại sao component re-render dù props không đổi?

Khi nào nên memoize (useMemo, useCallback, memo)?

Khi nào memoization trở thành over-optimization?

Debounce và Throttle trong React dùng để làm gì?

Virtualization (windowing) là gì?

Khi nào cần dùng virtualization (ví dụ danh sách 10k items)?

Cách phổ biến để tối ưu bundle size trong React app?

Redux hoạt động theo ba nguyên lý chính nào?

Flow dữ liệu cơ bản trong Redux: dispatch → reducer → store?

Tại sao reducer phải là pure function?

Immutability quan trọng như thế nào trong Redux?

Single source of truth trong Redux mang lại lợi ích gì?

Tại sao không nên lưu derived state (state tính toán được) trong store?

Redux khác Context API ở những điểm chính nào?

Khi nào nên dùng Redux, khi nào chỉ cần Context + useReducer?

Redux Thunk và Redux Saga khác nhau cơ bản như thế nào?

Backend-for-Frontend (BFF) pattern dùng khi nào?